

UNIVERSIDAD MAYOR DE SAN ANDRES

Carrera de Informatica

INFORME TECNICO

Actividades Individuales — Materia de Multimedia

Universitario:	Joan Jeremy Gonzales Acarapi
Docente:	Lic. Moises Silva
Materia:	Multimedia
Universidad:	Universidad Mayor de San Andres (UMSA)
Gestion:	2026

Actividad Individual a) — Clasificacion de Texturas

1. Descripcion General

La aplicacion implementa un sistema de clasificacion automatica de texturas en imagenes digitales. El algoritmo analiza cada pixel individualmente y lo asigna a una de cuatro categorias: Césped, Tierra, Cemento o Asfalto. Este enfoque es equivalente al utilizado en sistemas de clasificacion de imagenes satelitales.

La interfaz grafica fue desarrollada en Python utilizando Tkinter para la visualizacion y PIL/Pillow junto con NumPy para el procesamiento de imagenes a nivel de pixel.

2. Herramientas y Tecnologias Utilizadas

Tecnologia	Uso
Python 3.9+	Lenguaje de programacion principal
Pillow (PIL)	Carga, redimension y exportacion de imagenes
NumPy	Operaciones matriciales y calculo de varianza
Tkinter	Interfaz grafica de escritorio (GUI)
ttk (Tkinter)	Barra de progreso y componentes avanzados de UI
threading	Procesamiento en hilo secundario para no bloquear la UI

3. Metodologia de Clasificacion

3.1 Conversion al Espacio de Color HSV

Cada pixel RGB de la imagen se convierte al espacio de color HSV (Hue, Saturation, Value). Esta conversion permite separar el tono cromatico (H) de la luminosidad (V), mejorando la robustez del algoritmo ante variaciones de iluminacion. La formula aplicada es:

```
H = tono en [0, 360], S = saturacion en [0, 1], V = brillo en [0, 1]
```

3.2 Varianza Local 3x3

Para cada pixel se calcula la varianza de intensidad dentro de su vecindario inmediato de 3x3 pixeles. Este valor es el descriptor de textura: superficies rugosas o irregulares (cesped, tierra) presentan varianzas altas, mientras que superficies homogeneas (asfalto, cemento) presentan varianzas bajas.

```
varianza[i,j] = np.var( ventana_3x3_centrada_en(i,j) )
```

3.3 Clasificacion por Puntaje Combinado

Cada pixel recibe puntajes segun dos criterios independientes:

- Criterio 1 — Rango HSV: se verifica si H, S y V caen dentro de los rangos calibrados para cada clase. Por cada componente dentro del rango, el puntaje aumenta en 1 (maximo 3 puntos por clase).

- Criterio 2 — Varianza local: se añade un punto extra a las clases cuya textura esperada coincide con el nivel de varianza calculado (alta para cesped/tierra, baja para cemento/asfalto).

La clase con mayor puntaje acumulado (mínimo 2 puntos) es asignada al pixel. Si ninguna clase supera 1 punto, el pixel se clasifica como 'Otro'.

4. Rangos HSV de Referencia

Clase	Rango H
Césped	H: 55-175 S: 0.12-1.0 V: 0.08-0.88
Tierra	H: 8-42 S: 0.18-0.88 V: 0.12-0.78
Cemento	H: 0-360 S: 0.00-0.18 V: 0.38-0.94
Asfalto	H: 0-360 S: 0.00-0.32 V: 0.00-0.36

5. Arquitectura del Software

El proyecto sigue una arquitectura modular separando la lógica de procesamiento de la interfaz gráfica:

```
clasificador_texturas/
  main.py           <- Interfaz grafica (Tkinter)
  requirements.txt   <- Dependencias del proyecto
  README.md         <- Documentacion de instalacion y uso
  assets/
    images/
      ejemplo_cesped.jpg
      ejemplo_tierra.jpg
      ejemplo_cemento.jpg
      ejemplo_asfalto.jpg
  src/
    __init__.py
    clasificador.py  <- Logica pura de clasificacion
```

6. Funcionalidades de la Aplicación

- Carga de imagen propia mediante diálogo de archivo (JPG, PNG, BMP, WEBP).
- Carga de imágenes de ejemplo predefinidas (Césped, Tierra, Cemento, Asfalto) desde la carpeta assets/images/.
- Selector de clase a visualizar: Todas las clases o una clase específica. Al seleccionar una clase, los píxeles de las demás clases se muestran en escala de grises atenuada para destacar la clase elegida.
- Barra de progreso en tiempo real durante el procesamiento.
- Panel de estadísticas: porcentaje de píxeles detectados por clase.
- Exportación del mapa de texturas resultante en formato PNG o JPG.

7. Mapa de Colores del Resultado

Clase	Color en el mapa
Cesped	Verde RGB(34, 139, 34)
Tierra	Marron RGB(139, 90, 43)
Cemento	Gris RGB(180, 180, 180)
Asfalto	Negro RGB(50, 50, 50)
Otro	Morado RGB(190, 90, 190)

8. Instrucciones de Ejecucion

Instalacion de dependencias

```
pip install pillow numpy
```

Ejecutar la aplicacion

```
cd a-clasificadorTexturas  
python main.py
```

9. Conclusiones del Ejercicio a)

La implementacion del clasificador de texturas demuestra como el analisis combinado de informacion cromatica (HSV) y estructural (varianza local) permite diferenciar tipos de superficie de forma efectiva. El enfoque de clasificacion por puntaje ponderado resulta mas robusto que usar solo el color, ya que reduce los falsos positivos en condiciones de iluminacion variable.

La arquitectura modular facilita la extension del sistema, permitiendo agregar nuevas clases simplemente definiendo sus rangos HSV y comportamiento de textura en el modulo clasificador.py sin modificar la interfaz grafica.

Actividad Individual b) — Filtro de Suavizado 3x3

1. Descripcion General

La aplicacion implementa un filtro de promedio con ventana 3x3 aplicado directamente a nivel de pixel. El objetivo del filtro es reducir el ruido presente en imagenes digitales suavizando las transiciones abruptas de color entre pixeles vecinos, mejorando la calidad visual percibida.

El programa permite aplicar el filtro de forma iterativa (1, 2 o 3 pasadas) y visualizar simultaneamente la imagen original, la imagen suavizada y una imagen de diferencia amplificada que muestra exactamente el ruido eliminado.

2. Herramientas y Tecnologias Utilizadas

Tecnologia	Uso
Python 3.9+	Lenguaje de programacion principal
Pillow (PIL)	Carga, conversion y exportacion de imagenes
NumPy	Operaciones matriciales, padding y calculo de metricas
Tkinter	Interfaz grafica de escritorio
threading	Procesamiento paralelo para mantener la UI responsiva

3. Fundamento Matematico del Filtro

3.1 Filtro de Promedio (Box Filter)

El filtro de promedio es el filtro lineal de suavizado mas simple. Para cada pixel (i, j) de la imagen, su nuevo valor se calcula como el promedio aritmetico de los 9 pixeles en su vecindario de 3x3:

```
pixel_nuevo[i, j] = (1/9) * SUM( pixel[i+di, j+dj] )   para di, dj en {-1, 0, 1}
```

Este proceso se aplica de forma independiente para cada uno de los tres canales de color (R, G, B).

3.2 Manejo de Bordes — Padding Espejo

Los pixeles ubicados en los bordes de la imagen no tienen vecindario completo. Para evitar artefactos en los extremos, se aplica padding espejo (reflect): los bordes se extienden replicando especularmente los pixeles adyacentes antes de aplicar el filtro.

```
img_pad = np.pad(imagen, ((1,1), (1,1), (0,0)), mode='reflect')
```

3.3 Aplicacion en Multiples Pasadas

Cada pasada toma como entrada el resultado de la anterior. Matematicamente, aplicar el filtro de promedio 3x3 tres veces consecutivas es equivalente a una aproximacion del filtro gaussiano, produciendo un suavizado mas natural que una sola aplicacion con ventana mayor.

4. Metricas de Calidad Implementadas

Metrica	Descripcion y formula
MSE (Error Cuadratico Medio)	Mide la magnitud promedio del cambio. $MSE = \text{mean}(\text{original} - \text{suavizada})^2$. Valores altos indican mayor modificacion.
MAE (Error Absoluto Medio)	Promedio de cambio por pixel. $MAE = \text{mean}(\text{original} - \text{suavizada})$. Interpretable directamente en niveles de gris (0-255).
PSNR (Pico Senal-Ruido)	Calidad percibida en dB. $PSNR = 10 * \log_{10}(255^2 / MSE)$. Valores mayores a 30 dB indican buena calidad visual.
Pixeles modificados	Porcentaje de pixeles cuyo cambio supera 5 niveles de intensidad. Indica la cobertura del efecto de suavizado.

5. Imagen de Diferencia Amplificada

Para visualizar el ruido eliminado por el filtro se calcula la imagen de diferencia amplificada x5:

```
diferencia[i,j] = clip( |original[i,j] - suavizada[i,j]| * 5, 0, 255 )
```

La amplificacion x5 hace visibles diferencias que de otro modo serian imperceptibles al ojo humano (menores a 50 niveles de intensidad), facilitando la evaluacion visual del efecto del filtro.

6. Arquitectura del Software

```
filtro_suavizado/
  main.py                <- Interfaz grafica (Tkinter)
  requirements.txt
  README.md
  assets/
    images/
      ejemplo_rostro.jpg  <- Rostro sintetico con ruido gaussiano
      ejemplo_ciudad.jpg  <- Paisaje urbano con ruido gaussiano
  src/
    __init__.py
    filtro.py             <- Logica del filtro y metricas
```

7. Funcionalidades de la Aplicacion

- Carga de imagen propia o de ejemplos predefinidos (Rostro, Ciudad).
- Selector de pasadas: 1 pasada (suavizado leve), 2 pasadas (moderado) o 3 pasadas (fuerte).
- Visualizacion simultanea en tres paneles: Antes (original), Despues (suavizado) y Diferencia x5.
- Calculo y visualizacion de metricas MSE, MAE, PSNR y porcentaje de pixeles modificados.
- Exportacion de la imagen suavizada y la imagen de diferencia como archivos PNG/JPG.

- Procesamiento en hilo secundario con barra de progreso para mantener la interfaz responsiva.

8. Instrucciones de Ejecucion

Instalacion de dependencias

```
pip install pillow numpy
```

Ejecutar la aplicacion

```
cd b-filtroSuavizado  
python main.py
```

9. Conclusiones del Ejercicio b)

La implementacion del filtro de promedio 3x3 demuestra el principio fundamental del procesamiento de señales: el suavizado espacial reduce el ruido a costa de cierta perdida de detalle (bordes y contornos). La comparacion entre pasadas permite observar este compromiso: con 1 pasada se elimina ruido puntual manteniendo bordes definidos; con 3 pasadas el suavizado es mas intenso pero los bordes se difuminan notablemente.

La imagen de diferencia amplificada es especialmente util para el analisis tecnico, ya que revela exactamente que informacion fue modificada por el filtro, permitiendo evaluar su efectividad de forma visual e intuitiva.

Actividad Individual c) — Cover Multimedia: La Vaca Lola

1. Descripcion General del Proyecto

El presente trabajo corresponde a la Actividad Individual c) de la materia de Multimedia, cuyo objetivo es realizar una produccion multimedia de la cancion infantil 'La Vaca Lola', adoptando la identidad visual de un artista conocido, y vinculando el audio con una animacion donde el ritmo y el movimiento coincidan de forma sincronizada.

Se desarrollo una aplicacion de escritorio completamente funcional en Python utilizando la biblioteca Pygame como motor de renderizado. La aplicacion integra animacion generativa, reproduccion de audio sincronizada, y un video generado con Inteligencia Artificial que se proyecta en bucle en el centro de la pantalla.

2. Herramientas de Inteligencia Artificial Utilizadas

2.1 ChatGPT — Generacion de la Imagen Base

Se utilizo ChatGPT con capacidades de generacion de imagenes para crear la imagen ilustrativa del personaje principal de la cancion. Esta imagen sirvio como punto de partida visual y como insumo para la generacion del video.

Herramienta	ChatGPT (OpenAI)
Uso	Generar una imagen original relacionada con el concepto de La Vaca Lola
Ventaja	Permite crear contenido visual personalizado sin fotografia real ni modelado 3D

2.2 Wav2Lip — Sincronizacion Labial del Video

La imagen generada por ChatGPT fue procesada a traves de Wav2Lip (wav2lip.org), una herramienta de IA especializada en sincronizacion de movimiento labial (lip sync) a partir de una imagen estatica y un fragmento de audio.

Herramienta	Wav2Lip (wav2lip.org)
Entrada	Imagen generada por ChatGPT + fragmento de audio de La Vaca Lola
Salida	Video MP4 de 10 segundos con sincronia labial
Limitacion	La version gratuita restringe la duracion a 10 seg; el video se reproduce en bucle durante toda la cancion (2 min 24 seg)

2.3 Suno AI — Generacion de la Letra Musical

Se empleo Suno AI para obtener la letra de la cancion en el estilo deseado. Esta letra fue integrada al sistema de sincronizacion de la aplicacion, mostrando cada verso en el momento exacto de la reproduccion.

Herramienta	Suno AI
-------------	---------

Uso específico	Generacion y estructuracion de la letra de la cancion
Integracion	Los versos fueron incorporados a letras.py con marcas de tiempo en segundos

3. Arquitectura del Software

La aplicacion fue desarrollada en Python con arquitectura modular organizada por responsabilidad:

```
vaca_lola/
  main.py                <- Punto de entrada
  config.py              <- Constantes globales (colores, FPS,
  BPM)
  README.md
  src/
    escena.py            <- Controlador principal y bucle de juego
    audio/
      audio.mp3          <- Cancion original
      gestor_audio.py    <- Carga y control de pygame.mixer
      letras.py          <- Letra con timestamps en segundos
    assets/video/
      video.mp4          <- Video generado con Wav2Lip (10 seg,
  bucle)
    personajes/
      monigote.py        <- Personaje humano animado (stick-
  figure)
      vaca.py            <- Personaje vaca animado
    ui/
      pantalla_menu.py   <- Pantalla de menu inicial
      pantalla_show.py   <- Renderizado principal + video
      hud.py             <- HUD: barra de beat, estado, controles
      particulas.py      <- Sistema de particulas decorativas
    utils/
      anim.py            <- Funciones de animacion (rebote, pulso)
      draw.py            <- Funciones auxiliares de dibujo
```

4. Componentes Principales

4.1 Motor de Animacion (Pygame a 60 FPS)

La aplicacion corre a 60 FPS. Todos los elementos animados responden a una variable de tiempo continuo y a la intensidad del beat, calculada a partir del BPM de la cancion. Esto crea el efecto de que toda la animacion baila al ritmo de la musica.

4.2 Reproduccion de Video en Bucle (OpenCV + Pygame)

El video MP4 generado por Wav2Lip se reproduce dentro de la ventana principal utilizando OpenCV para decodificar los frames. Cada frame es convertido de formato BGR (OpenCV) a

RGB (Pygame) y renderizado como superficie en el centro de la pantalla. Al llegar al ultimo frame, el sistema reinicia automaticamente el video logrando un bucle fluido.

4.3 Sincronizacion de Audio, Video y Animacion

El modulo gestor_audio.py utiliza pygame.mixer para la reproduccion del audio. La sincronizacion de la letra se realiza mediante pygame.mixer.music.get_pos(), que devuelve la posicion exacta en milisegundos. Este valor se compara con los timestamps de letras.py para mostrar cada verso en el momento correcto.

4.4 Control de Pausa Global

Al presionar la barra espaciadora, la aplicacion entra en modo pausa global: se detiene el audio, se congela el tiempo interno de la animacion y se detiene el avance de frames del video. Al reanudar, todos los sistemas retoman exactamente desde donde se detuvieron.

5. Herramientas y Tecnologias Utilizadas

Tecnologia	Uso
Python 3.9+	Lenguaje principal
Pygame	Motor de renderizado, animacion y reproduccion de audio
OpenCV	Decodificacion y reproduccion de video MP4 frame a frame
NumPy	Conversion de frames BGR a RGB
ChatGPT	Generacion de la imagen base del personaje
Wav2Lip	Generacion del video con sincronia labial
Suno AI	Generacion de la letra musical con estructura temporal

6. Controles de la Aplicacion

Tecla	Accion
ESPACIO	Pausa / Reanuda todo (audio + animacion + video)
ESC	Cierra la aplicacion

7. Instrucciones de Ejecucion

Instalacion de dependencias

```
pip install pygame opencv-python numpy
```

Ejecutar la aplicacion

```
cd c-vaca_lola
python main.py
```

La aplicacion cargara automaticamente el audio desde src/audio/audio.mp3 y el video desde src/assets/video/video.mp4, iniciando la animacion de forma inmediata.

8. Conclusiones del Ejercicio c)

Este proyecto demuestra la integración efectiva de múltiples tecnologías multimedia en un único entorno interactivo. La combinación de animación generativa en tiempo real, sincronización precisa de audio y letra, y la incorporación de un video generado con Inteligencia Artificial logran una experiencia audiovisual coherente y dinámica.

El uso de herramientas como ChatGPT, Wav2Lip y Suno AI no solo simplifica la producción de contenido multimedia, sino que abre nuevas posibilidades creativas, permitiendo generar personajes animados cantantes a partir de una simple imagen y un fragmento de audio.